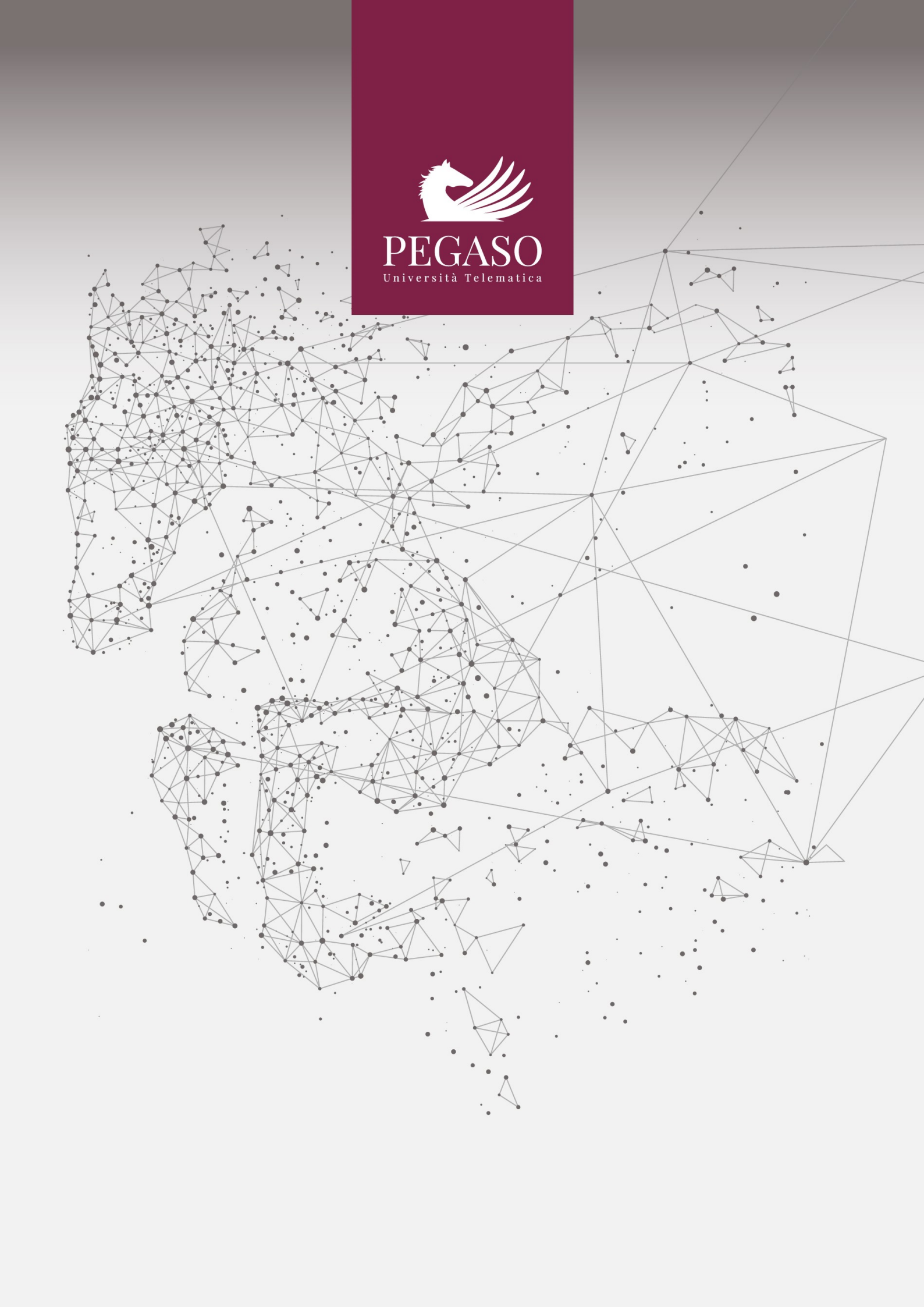

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. COMPILATORE E ASSEMBLER 3

2. LINKER E LOADER 7

3. BYTECODE E JAVA VIRTUAL MACHINE 9

BIBLIOGRAFIA 12

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Compilatore e Assembler

Il grande interesse per lo sviluppo dei Linguaggi di programmazione ad Alto Livello è stato motivato dai notevoli vantaggi che essi offrono che hanno determinato anche importanti progressi nell'ambito delle metodologie di progettazione del Software.

Il primo vantaggio apparso con chiarezza fin dai primi anni dello sviluppo dell'attività di programmazione è la possibilità di rappresentare gli algoritmi mediante un linguaggio con una sintassi che utilizza termini e simboli del linguaggio naturale, e con operazioni di base più vicine a quelle utilizzate per la progettazione di un algoritmo risolutivo di un problema. In questa ottica, è stato anche possibile progettare Linguaggi specifici adatti ad ambiti particolari.

Un ulteriore vantaggio immediatamente emerso è stata l'indipendenza dei programmi scritti in Linguaggio ad Alto Livello dal computer che li esegue, grazie ai programmi di traduzione che forniscono programmi equivalenti nel Linguaggio Macchina del computer, l'unico che la macchina è capace di comprendere, in cui le istruzioni sono rappresentate da sequenze binarie. Il Linguaggio Assembly consente una rappresentazione delle istruzioni strutturalmente vicina al Linguaggio Macchina, ma utilizzando simboli alfanumerici e la Notazione decimale per i numeri.

Ricordiamo che le tre classi di Linguaggi di programmazione:

- Linguaggio ad Alto Livello
- Linguaggio Assembly
- Linguaggio Macchina

costituiscono una GERARCHIA DI LINGUAGGI.

La traduzione delle istruzioni partendo dal livello più alto porta a discendere lungo la Gerarchia fino al livello più vicino alla macchina.

Ad esempio, la seguente istruzione in linguaggio C,:

```
f = (g + h) – ( i + j )  ;
```

può essere tradotta nelle seguenti tre istruzioni in Assembly MIPS:

```
add   $t0, $s1, $s2
add   $t1, $s3, $s4
sub   $s0, $t0, $t1
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

alle quali corrisponderanno altrettante istruzioni in Linguaggio Macchina MIPS:

00000010001100100100000000100000

00000010011101000100100000100000

00000001000010011000000000100010

Attualmente la programmazione in linguaggio Assembly è riservata ad applicazioni particolari, ed è utilizzata nei sistemi *embedded* allo scopo di ottimizzare il programma con tecniche che consentono di ridurre il numero di istruzioni risparmiando tempo di esecuzione e spazio di memorizzazione.

Parallelamente allo sviluppo dei Linguaggi ad Alto Livello, si è determinato un grande impegno nella realizzazione dei programmi di traduzione, che forniscono automaticamente la traduzione in Linguaggio Macchina.

Il processo di trasformazione di un programma scritto in un Linguaggio ad Alto Livello in un equivalente programma ESEGUIBILE in Linguaggio Macchina comprende delle operazioni di base che vanno effettuate per ottenere un programma che possa essere eseguito su una specifica piattaforma.

Gli attuali compilatori sono programmi molto complessi. Allo scopo di evidenziare le principali operazioni necessarie per ottenere il programma eseguibile, diamo una descrizione sintetica distinguendo quattro principali fasi.

Nella **prima fase** il programma in Linguaggio ad Alto Livello viene tradotto dal *COMPILATORE* nel **Linguaggio Assembly** relativo al computer che dovrà eseguirlo. Questo programma di traduzione è strettamente connesso al Linguaggio ad Alto Livello ed alla specifica Architettura del computer.

In una **fase successiva** il programma *ASSEMBLER* effettua l'ulteriore traduzione del programma in Linguaggio Assembly in un programma in **Linguaggio Macchina**, che viene scritta in un file denominato ***codice oggetto***.

Il programma Assembler mantiene traccia, anche se in forma non definitiva, delle relazioni tra simboli e indirizzi di Memoria per le variabili e le etichette utilizzate nei salti e nelle chiamate di procedura. Queste informazioni sono scritte nel file del *codice oggetto* insieme alla lista delle istruzioni in Linguaggio Macchina che fornisce la traduzione del programma.

I collegamenti tra simboli ed indirizzi di Memoria verranno precisati e resi definitivi attraverso le due ulteriori fasi descritte nel successivo paragrafo, che portano alla determinazione del programma eseguibile ed all'avvio dell'esecuzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I Compilatori detti ad un passo producono direttamente il *codice oggetto*. Un aspetto della ricerca riguarda il progetto di Compilatori che determinano *ottimizzazioni del codice oggetto* per ottenere una traduzione in Linguaggio Macchina che minimizzi il tempo di esecuzione e l'uso delle risorse di sistema.

NOTA

Poiché il Linguaggio Assembly costituisce un livello Software intermedio tra il programma in Linguaggio ad Alto Livello e la traduzione in Linguaggio Macchina eseguibile dall'Hardware del sistema, il programma di traduzione Assembler può anche accettare particolari istruzioni Assembly, dette **pseudoistruzioni**, che corrispondono a combinazioni di istruzioni Assembly.

Quindi una pseudoistruzione non è direttamente realizzata a livello Hardware, e viene eseguita mediante l'esecuzione di altre istruzioni.

L'inclusione di pseudoistruzioni nel progetto di un linguaggio Assembly consente di avere un insieme di istruzioni più ampio di quello effettivamente realizzato, senza dover parallelamente complicare il progetto Hardware.

L'uso delle pseudoistruzioni che sintetizzano combinazioni di istruzioni consente al programmatore di scrivere programmi più brevi e di aumentare la leggibilità del codice.

L'Assembly MIPS include alcune pseudoistruzioni che hanno una sintassi analoga a quella delle istruzioni implementate nel progetto Hardware.

Ad esempio, l'operazione *move* che consentente di copiare il contenuto di un registro in un altro registro, è realizzata in Assembly MIPS dalla pseudo istruzione:

```
move $t0, $t1
```

e corrisponde all'esecuzione della istruzione Assembly:

```
add $t0, $t1, $zero
```

La pseudoistruzione *move* fornisce solo il vantaggio di una maggiore leggibilità. Altre pseudoistruzioni offrono anche il vantaggio di una maggiore sintesi.

Ad esempio, l'operazione *branch on less then* (salta se strettamente minore) che effettua il salto condizionato se la relazione strettamente minore tra i contenuti di due registri è vera, è realizzata in Assembly MIPS dalla pseudoistruzione:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

blt $t0, $t1, *etichetta*

che sintetizza la combinazione delle due istruzioni Assembly MIPS:

```
slt $at, $t0, $t1          #  pone nel registro $at il valore 1
                           #  se il contenuto di $t0  è minore  del contenuto di
                           $t1
bne $at, $zero, etichetta  # salta a etichetta se il contenuto
                           # di $at è diverso da 0, cioè se la relazione minore
                           è vera
```

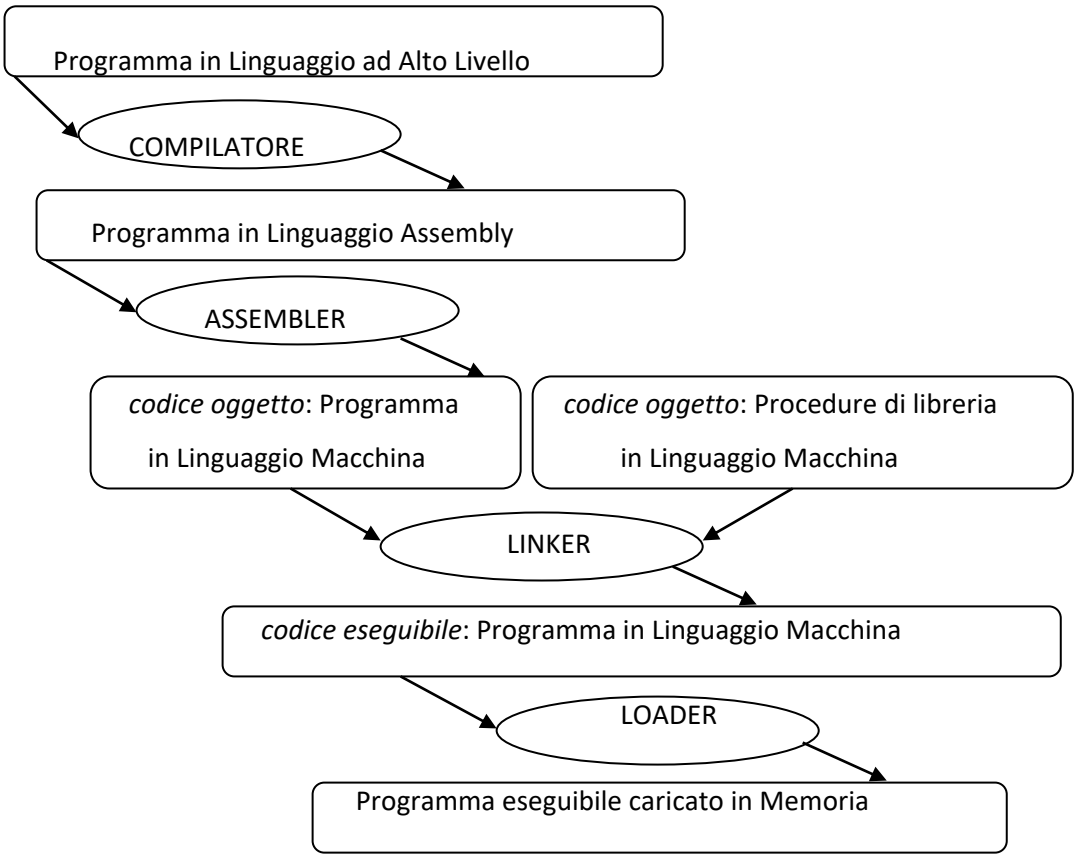
L'esempio mostra anche che per la traduzione delle pseudoistruzioni il programma Assembler utilizza come registro di supporto il registro di indirizzo $at, che corrisponde all'indirizzo 1. Questo registro è riservato all'Assembler.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Linker e Loader

Nella **terza fase** il programma *LINKER* (link editor) aggiorna gli indirizzi stabilendo gli opportuni collegamenti tra il *codice oggetto* del programma e il *codice oggetto* delle procedure da esso utilizzate, comprese quelle delle librerie standard, che sono state compilate separatamente. Il programma *Linker* produce il ***codice eseguibile***.

L'**ultima fase** è effettuata dal programma *LOADER* che carica il programma eseguibile nella Memoria del computer e provvede ad assegnare lo spazio necessario alle strutture dati e ad effettuare le operazioni di inizializzazione per avviare l'esecuzione delle istruzioni.



L'approccio alle quattro fasi descritte ha avuto nel tempo miglioramenti che hanno determinato modificazioni del processo che porta alla costruzione di un programma tradotto in Linguaggio Macchina che possa essere eseguito su una specifica piattaforma.

Il processo che è stato descritto è detto *statico*, in relazione al fatto che una libreria Software viene collegata al programma staticamente in fase di compilazione, cioè prima dell'esecuzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Una delle principali modifiche riguarda lo sviluppo di Compilatori basati sull'approccio delle *librerie a caricamento dinamico*, in base al quale le procedure e le funzioni di libreria vengono collegate al programma e caricate in Memoria solo quando sono chiamate nel corso dell'esecuzione del programma.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Bytecode e Java Virtual Machine

Java è un linguaggio di programmazione ad Alto Livello, basato sul paradigma *objec oriented*.

Annunciato ufficialmente nel 1995, inizialmente fu sviluppato da James Gosling e altri ingegneri presso Sun Microsystems, acquisita nel 2010 da Oracle Corporation, che attualmente detiene il marchio registrato.

Il Linguaggio Java e stato specificamente progettato per essere il più possibile indipendente dalla *piattaforma* di esecuzione (costituita dall'Architettura del computer e dal Sistema Operativo).

Per i programmi scritti utilizzando la maggior parte dei linguaggi di programmazione è necessario disporre del Compilatore relativo alla piattaforma reale su cui il programma verrà eseguito. Il Compilatore fornisce il file del *codice oggetto* strutturato in funzione delle caratteristiche Hardware di tale piattaforma.

Nel caso dei programmi scritti in Linguaggio Java il Compilatore crea un file con estensione *.class* che contiene la traduzione del programma Java in un Linguaggio denominato ***bytecode***, che è relativo ad una **macchina astratta** denominata ***Java Virtual Machine***.

Per ESEGUIRE il programma in *bytecode* su una specifica piattaforma è necessario disporre di un **programma che emula** il comportamento della *Java Virtual Machine*, detto ***Interprete* Java**.

L'*Interprete* Java esamina ogni istruzione in *bytecode* e la esegue mediante istruzioni in Linguaggio Macchina della specifica piattaforma, ottenendo così l'esecuzione delle operazioni corrispondenti all'istruzione in *bytecode*.

Quindi, il vantaggio sta nel fatto che per eseguire un programma in *bytecode* non è necessario disporre del corrispondente Hardware capace di eseguire tali istruzioni.

Un qualunque Hardware può eseguire istruzioni in *bytecode* mediante il programma *Interprete* Java, che può essere considerato una implementazione Software della *Java Virtual Machine*, la **macchina astratta** capace di eseguire istruzioni in *bytecode*.

Esistono implementazioni gratuite di programmi *Interpreti* Java per tutte le principali piattaforme attualmente in uso, e per particolari ambienti come telefoni cellulari e palmari.

Questa caratteristica di portabilità di un programma in *bytecode* Java è espressa dal motto "*write once, run anywhere*" (scrivi una volta, esegui ovunque) riferito al fatto che il *bytecode* prodotto dalla compilazione (scrivi una volta) può essere eseguito su un qualunque computer mediante un *Interprete* Java (esegui ovunque).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un ulteriore aspetto connesso alla portabilità di Java è la possibilità di scrivere un tipo di programma in Java, detto *Applet*, progettato per essere eseguito direttamente nel World Wide Web, poiché il *bytecode* di questo tipo di programma, inserito in una pagina web, può essere eseguito all'interno di un *Web browser*.

Quindi, in sintesi, un programma in *bytecode* è la rappresentazione del relativo programma scritto in Linguaggio Java così come un programma Assembly lo è per un programma scritto in un altro Linguaggio ad Alto Livello, con la differenza fondamentale che un programma Assembly è legato ad una specifica Architettura concreta, mentre un programma in *bytecode* è legato ad una Architettura astratta emulabile da un programma Software su qualunque Architettura disponibile.

Un'istruzione in *bytecode* è formata da un *Codice Operativo* che identifica l'istruzione e che coinvolge eventuali operandi richiesti dall'operazione relativa all'istruzione.

Il *bytecode* è così chiamato perché i *Codici Operativi* occupano un solo byte, anche se la lunghezza dell'intera istruzione può variare perché ogni operazione utilizza uno specifico numero di operandi.

Alcuni codici operativi hanno un prefisso costituito da una lettera, come ad esempio *aload* e *iload*, che indica il Tipo di dati che è coinvolto nell'operazione.

Il prefisso "a" indica che l'operazione utilizza il riferimento ad un oggetto, il prefisso "i" che utilizza un intero. Altri Codici Operativi hanno i prefissi "b" per byte, "c" per carattere, "d" per double, ecc.

Una descrizione completa della Sintassi del *bytecode* Java richiede la descrizione dell'Architettura della Java Virtual Machine, che coinvolge la gestione di struttura dati dei Tipi *stack* e *array*, utilizzate per memorizzare le informazioni necessarie per eseguire le istruzioni in *bytecode* e per controllarne l'ordine di esecuzione.

A titolo puramente indicativo della gerarchia di Linguaggi costituita dal Linguaggio ad Alto Livello Java, dalla traduzione in *bytecode* e dalla rappresentazione binaria in sequenze di byte, diamo un esempio molto semplice considerando il **metodo Java**:

```
public String employeeName()
{
return name;
}
```

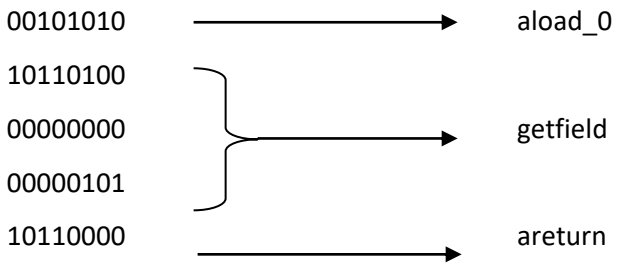
La relativa traduzione in ***bytecode* Java** è data da:

```
aload_0
getfield
areturn
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

che corrisponde alla **rappresentazione binaria in byte**:



Le due istruzioni *aload_0* e *areturn* non hanno operandi e sono tradotte solo con i rispettivi *Codici Operativi*, invece la traduzione della seconda istruzione *getfield* con i suoi operandi richiede tre byte (uno per il *Codice Operativo* e due per gli operandi).

Per maggiori dettagli si può consultate l'articolo da cui è tratto l'esempio:

https://www.ibm.com/developerworks/library/it-haggar_bytecode/index.html.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.
- ORACLE Java Documentation, https://docs.oracle.com/javase/10/
- P. Haggar, “Understanding bytecode makes you a better programmer”, https://www.ibm.com/developerworks/library/it-haggar_bytecode/index.html

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d’autore (L. 22.04.1941/n. 633).